UNIVERSITY OF LAHORE

Faculty of Information Technology

Course: Artificial Intelligence

Assignment 2 Title: Customer Churn Prediction Web Application Using Flask
Based on: Assignment 1 - Customer Churn Prediction Using Machine Learning
Dataset: `WA_Fn-UseC_-Telco-Customer-Churn.xlsx`
Deployment Platform: PythonAnywhere (`pythonanywhere.com`)
Submission: Complete Flask Project + Live URL + Final Report (`.pdf`)

Objective

The purpose of this assignment is to convert the machine learning work completed in Assignment 1 into a working web application. You will save the final trained churn prediction model, build a simple user interface using HTML and CSS, connect it to a Flask backend, and deploy the complete application on PythonAnywhere.

The final website must allow a user to enter customer information and receive a clear Churn or No Churn prediction from your trained model. This assignment continues directly from Assignment 1, so you should reuse your own preprocessing steps, selected features, and best-performing model.

Tasks

Complete the following application and deployment pipeline:

1. Carry Forward the Final Model: Use the best-performing classification model selected in Assignment 1. Make sure the same preprocessing, encoding, scaling, and feature order used during training are also used when new customer data is submitted through the website.
2. Save the Trained Model: Save the final trained model using a suitable method such as `joblib` or `pickle`. Save any required encoder, scaler, preprocessing object, or feature information that is needed to make correct predictions later.
3. Create the Flask Project: Create a Flask application with a clear project structure. Your project should contain the Flask backend file, a `templates` folder for HTML pages, a `static` folder for CSS, the saved model files, and any other supporting files required to run the application.
4. Build the Frontend Using HTML and CSS: Create a clean and simple web page containing a customer input form. The form should collect the important customer features required by your trained model. Use proper labels, suitable input types or dropdowns, readable spacing, and basic CSS styling so the page is easy to use.
5. Connect the Frontend with Flask: Create Flask routes that display the form, receive the submitted customer values, convert them into the correct format, apply the same preprocessing used during model training, and pass the final input to the saved model.

6. Display the Prediction Result: After the user submits the form, show a clear prediction on the web page, for example Churn or No Churn. The result should be understandable to a normal user and should not display raw arrays or technical model output.
7. Input Validation and Error Handling: Handle incorrect, empty, or unexpected inputs in a simple and user-friendly way. The application should not crash when a user submits incomplete data. Use meaningful messages where appropriate.
8. Test the Application Locally: Run the Flask application on your own computer before deployment. Test multiple sample customers, including at least one expected churn case and one expected no-churn case. Confirm that the web prediction is consistent with your machine learning model.
9. Deploy on PythonAnywhere: Create or use a PythonAnywhere account and deploy the Flask application there. Upload the project files, install the required Python packages, configure the web application and WSGI file correctly, and reload the application. The final URL must open from another device or browser without using your local computer.
10. Final Online Testing: Open the deployed PythonAnywhere URL and test the complete flow again. Check the page design, form submission, preprocessing, prediction result, and error handling. Fix any deployment or path issues before submission.

Minimum Project Structure

```
project/
  app.py
  model.pkl or model.joblib
  requirements.txt
  templates/
    index.html
  static/
    style.css
```

You may add extra Python files, model artifacts, images, or templates if your own solution requires them.

Final Report

Submit an updated PDF report that continues your Assignment 1 work. Include screenshots of saving the trained model, the project folder structure, the HTML/CSS interface, important Flask backend code, local execution, prediction results, PythonAnywhere configuration, and the final deployed website. For each major screenshot, briefly explain what was done, why it was needed, and what the result shows.

Required Submission

Submit (1) the complete Flask project folder or ZIP file, (2) the saved trained model and all required preprocessing files, (3) the final PDF report with screenshots and explanations, and (4) the working PythonAnywhere URL. Your deployed application must be accessible at the time of checking. Marks will be based on correct model integration, frontend usability, Flask implementation, successful deployment, prediction correctness, and understanding of your own code.

Important: The application must use your Assignment 1 machine learning model. Static HTML, screenshots, generated code, or hard-coded predictions will not be accepted. You must be able to explain the complete input-to-prediction flow.